COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010-Data Structure and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	10/6/2026	**DATE SUBMITTED:**	10/6/2026
NAME:	Michael Yuri D. Barbo Wayne Kaisen C. Paraiso	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 11.1 **Basic Algorithm Analysis**

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

Performing worst case analysis:

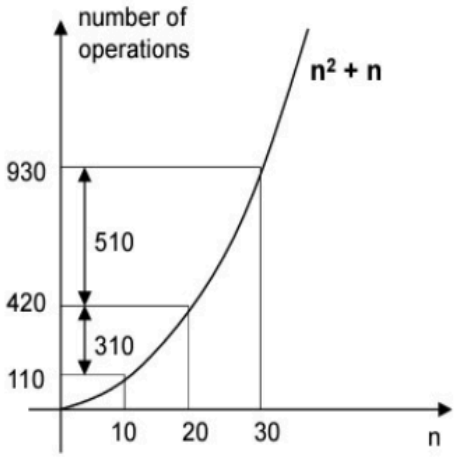
- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Analysis:

- The worst case happens when the lop never exits early. In diff, the early return fires as soon as search finds a match (!= -1). So the worst case is when no element of y appears in x, every search scans a;; of x without finding anything, and the loop runs all n iterations before returning true

Given the above information, the total number of comparisons in the worst case is: m * n
Assume m = n: n^2

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:

- The worst case happens when no elements of array y are found in array x. In this situation, search never returns a match, so the early return false never executes. The loop in diff runs all n iterations, and each call to search scans all m elements of x before failing. With m = n, the total work is n * n = n^2 comparisons, plus about n for the loop overhead, giving n^2 + n operations

CODE

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>
#include <cmath>

//Step 1 and Step 2: Define the Student class and overload comparison operators
class Student {
private:
    std::pair<int, int> name;
    bool vaccinated;

public:
    //Constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v) {}

    //Getters
    auto get_name() const { return name; }
    auto is_vaccinated() const { return vaccinated; }

    //Two people are same if they have the same name
    bool operator==(const Student& p) const {
        return this->name == p.name;
    }
```

```
27
28      //The ordering of a set of people is defined by their name
29      bool operator<(const Student& p) const {
30          return this->name < p.name;
31      }
32
33      bool operator>(const Student& p) const {
34          return this->name > p.name;
35      }
36  };
37
38  //Step 3: Function to generate a student with random credentials
39  auto generate_random_Student(int max) {
40      std::random_device rd;
41      std::mt19937 rand(rd());
42
43      //The IDs of Student should be in range [1, max]
44      std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);
45
46      //Generate random credentials
47      auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
48      bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
49
50      return Student(random_name, is_vaccinated);
51  }
52
53  //Step 5: Binary search logic to check if vaccination is needed
54  bool needs_vaccination(Student P, std::vector<Student>& people) {
55      auto first = people.begin();
56      auto last = people.end();
57
58      while (true) {
59          auto range_length = std::distance(first, last);
60          if (range_length <= 0) return true;
61
62          auto mid_element_index = std::floor(range_length / 2);
63          auto mid_element = *(first + mid_element_index);
64
65          //Return true if the Student is found in the sequence and he/she's not vaccinated
66          if (mid_element == P && mid_element.is_vaccinated() == false)
67              return true;
68          else if (mid_element == P && mid_element.is_vaccinated() == true)
69              return false;
70          else if (mid_element > P)
```

```
            std::advance(last, -mid_element_index);
        else if (mid_element < P)
            std::advance(first, mid_element_index);

        //Student not found in the sequence and therefore should be vaccinated
        if (range_length == 1)
            return true;
    }
}

//Function to perform search, measure execution time, and display results
void search_test(int size, Student p) {
    std::vector<Student> people;

    //Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());

    //To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();

    bool search_result = needs_vaccination(p, people);

    //Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();

    std::cout << "Time taken to search "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds" << std::endl;

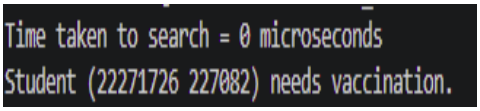
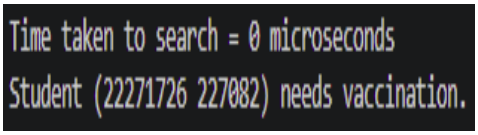
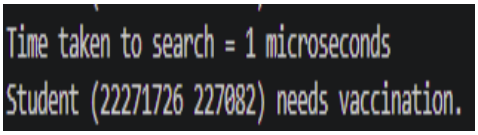
    if (search_result)
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") needs vaccination." << std::endl;
    else
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") does not need vaccination." << std::endl;
}
```

```
//Step 6: Driver code
int main() {
    // Generate a Student to search
    auto p = generate_random_Student(100000000);

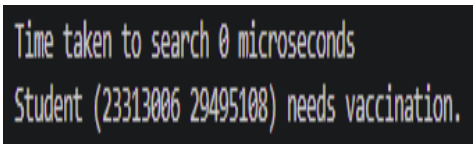
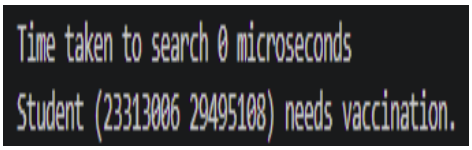
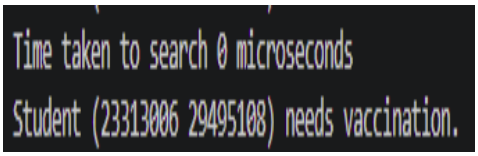
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);

    return 0;
}
```

ILO B Paraiso:

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	0 Microsecond		The code took 0 microsecond to search with an input size of 1,000
10000	0 Microsecond		The code took 0 microsecond to search with an input size of 10,000
100000	1 Microsecond		The code took 1 microsecond to search with an input size of 100,000

ILO B Barbo:

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	0 Microsecond		The search took around 0 microsecond to find the student who needed vaccinations with an input size of 1,000.
10000	0 Microsecond		The search took around 0 microsecond to find the student who needed vaccinations with an input size of 10,000.
100000	0 Microsecond		The search took around 0 microsecond to find the student who needed vaccinations with an input size of 100,000.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

For each of the problems below, provide the following:

- Theoretical Analysis
- Experimental Analysis
- Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

CODE

```cpp
#include <iostream>
#include <chrono>

using namespace std;

bool unique(int A[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (A[i] == A[j]) return false;
    return true;
}

int main() {
    int sizes[] = {1000, 10000, 50000, 100000};
    for (int n : sizes) {
        int* A = new int[n];
        for (int k = 0; k < n; k++) A[k] = k;   // all distinct = worst case

        auto s = chrono::high_resolution_clock::now();
        unique(A, n);
        auto e = chrono::high_resolution_clock::now();

        cout << "n=" << n << "  time="
             << chrono::duration_cast<chrono::microseconds>(e - s).count()
             << " microseconds\n";

        delete[] A;
    }
}
```

Supp Problem 1 Barbo:

- **Theoretical Analysis:**

  - This algorithm checks whether the elements of an array are distinct. The basic operation is the comparison A[i] equals A[j]. The worst case happens when all elements are distinct, so the early return never executes and both loops run to completion. The outer loop runs from i = 0 to n-2, and for each i, the inner loop performs n - 1 - i comparisons. In total, this is (n - 1) + (n - 2) + 1 = n(n - 1)/2 comparisons, so the worst-case time complexity is $O(n^2)$. The best case is when A[0] = A[1], which takes one comparison, or O(1).

- **Experimental Analysis:**

```
n=1000  time=337 microseconds
n=10000  time=33755 microseconds
n=50000  time=875832 microseconds
n=100000  time=3463981 microseconds
```

- **Analysis and Comparison:**

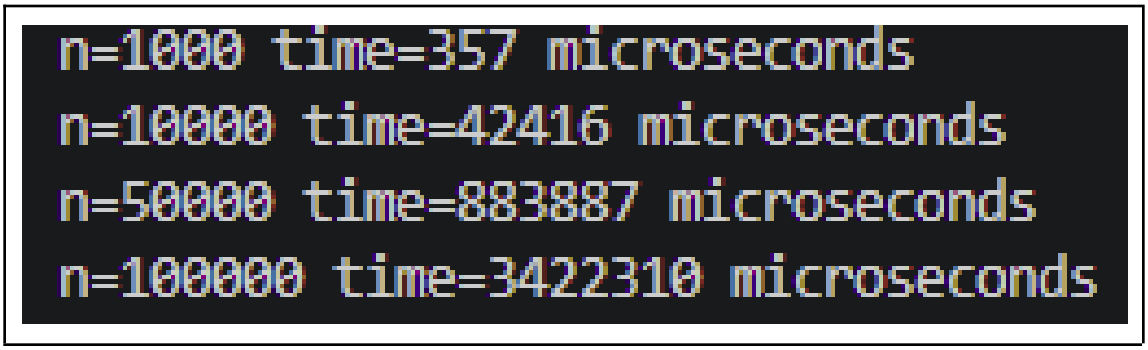
  - The theoretical analysis predicts $O(n^2)$, so when n grows by a factor k, the time should grow by about k^2. The experimental results match this. When n increased 10× (1,000 to 10,000), the time increased about 100× (337 to 33,755 microseconds). When n doubled (50,000 to 100,000), the time increased about 4× (875,832 to 3,463,981 microseconds). Small differences come from hardware and background processes. This confirms that the worst case of the algorithm is $\Theta(n^2)$.

Supp Problem 1 Paraiso:

- **Theoretical Analysis:**

  - This is a uniqueness algorithm that checks each element of the array if the elements are distinct or similar. The basic operation of the algorithm if the comparison. The worst case happens when all elements of the array are distinct, or when the only duplicate is the last pair, so the early return false never executes and both loops run to completion. This code runs from i = 0 to n-2, and for each value of i, the inner loop runs from j = i + 1 to n - 1, which means it performs n-1-i comparisons.

- **Experimental Analysis:**



- **Analysis and Comparison:**

  - The algorithm checks whether all array elements are distinct by comparing each element with every element after it. In the worst case, both loops run to completion and perform n(n-1)/2 comparisons, giving a time complexity of Θ(n²). The measured times (357 µs at n=1,000 up to 3,422,310 µs at n=100,000) confirm this: when n grew 100×, theory predicts a 10,000× increase in time and the measured increase was about 9,587×, within roughly 4%. The cost per comparison also stayed nearly constant at about 0.7 to 0.85 ns, as expected for a quadratic algorithm. Small deviations come from cache behavior, compiler optimizations, and timer noise. Overall, the results show the algorithm is quadratic and impractical for large inputs, where sorting first (O(n log n)) or using a hash set (O(n) on average) would be far better.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

CODE

```cpp
#include <iostream>
#include <chrono>
#include <iomanip>

long long mult_count = 0;

unsigned long long rpower(unsigned long long x, int n) {
    if (n == 0) return 1;
    mult_count++;
    return x * rpower(x, n - 1);
}

unsigned long long brpower(unsigned long long x, int n) {
    if (n == 0) return 1;
    if (n % 2 == 1) {
        unsigned long long y = brpower(x, (n - 1) / 2);
        mult_count += 2;
        return x * y * y;
    } else {
        unsigned long long y = brpower(x, n / 2);
        mult_count += 1;
        return y * y;
    }
}

template <typename F>
double time_us(F f, int x, int n, int reps, long long &mults) {
    mult_count = 0;
    volatile unsigned long long sink = 0;
    auto begin = std::chrono::steady_clock::now();
    for (int i = 0; i < reps; i++) sink = f(x, n);
    auto end = std::chrono::steady_clock::now();
    mults = mult_count / reps;
    return std::chrono::duration<double, std::micro>(end - begin).count()
           / reps;
}

```

```cpp
int main() {
    const int x = 3;

    std::cout << "3^10: rpower = " << rpower(x, 10)
              << ", brpower = " << brpower(x, 10) << "\n\n";

    std::cout << std::left << std::setw(10) << "n"
              << std::setw(16) << "rpower mults"
              << std::setw(18) << "rpower time(us)"
              << std::setw(16) << "brpower mults"
              << std::setw(18) << "brpower time(us)" << "\n";

    int sizes[] = {10, 100, 1000, 5000, 10000, 20000};
    for (int n : sizes) {
        long long rm, bm;
        double rt = time_us(rpower, x, n, 1000, rm);
        double bt = time_us(brpower, x, n, 1000, bm);
        std::cout << std::left << std::setw(10) << n
                  << std::setw(16) << rm
                  << std::setw(18) << std::fixed << std::setprecision(3)
                  << rt
                  << std::setw(16) << bm
                  << std::setw(18) << bt << "\n";
    }
    return 0;
}
```

Supp Problem 2 Barbo:

- **Theoretical Analysis:**

  - The rpower does one multiplication per call and recurses on n-1, so it makes exactly n multiplications. That's Θ(n) time. brpower halves n each call, doing 1 or 2 multiplications per level, so it takes about $\log_2 n$ levels. That's Θ(log n) time.

- **Experimental Analysis:**

```
3^10: rpower = 59049, brpower = 59049

n         rpower mults    rpower time(us)   brpower mults   brpower time(us)
10        10              0.022             6               0.013
100       100             0.275             10              0.022
1000      1000            2.942             16              0.033
5000      5000            14.675            18              0.040
10000     10000           29.542            19              0.043
20000     20000           64.525            20              0.092
```

- **Analysis and Comparison:**

  - The experiments match the theory. rpower's multiplication count equals n exactly, and its time grows roughly linearly. brpower's count barely moves while n goes from 10 to 20000, and its time stays under 0.05 µs. At n = 20000, brpower does about 1000 times fewer multiplications and runs about 2000 times faster. So the repeated-squaring approach beats the simple one, and the gap gets bigger as n grows.

Supp Problem 2 Paraiso:

- **Theoretical Analysis:**

  - rpower computes x^n by peeling off one factor per call, so it does exactly n multiplications and uses n stack frames, making it Θ(n) in both time and space. brpower halves the exponent on each call, doing one multiplication per level plus one extra for each odd step, for a total of $\lfloor \log_2 n \rfloor + 1 + \text{popcount}(n)$, which is Θ(log n) in both time and space (best case n a power of 2, worst case n = 2^k − 1, but both are logarithmic). For example, at n = 10000 rpower needs 10000 multiplications while brpower needs only 19, and doubling n doubles rpower's work but adds just one multiplication for brpower, so brpower is exponentially faster. Results overflow 64 bits once 3^n exceeds that range (n > 40), but the operation counts remain valid.

- **Experimental Analysis:**

n	rpower mults	rpower time(us)	brpower mults	brpower time(us)
10	10	0.000	6	0.000
100	100	0.000	10	0.000
1000	1000	4.003	16	0.000
5000	5000	17.997	18	0.000
10000	10000	35.512	19	0.000
20000	20000	78.009	20	0.998

- **Analysis and Comparison:**

  - My results confirm the theory. The multiplication counts match Θ(n) for rpower and Θ(log n) for brpower exactly, and rpower's timing scales linearly at about 4 ns per multiplication. brpower's timings were too fast for my timer to resolve, so I relied on the multiplication counts for the comparison. To measure real times for it, I would increase the repetitions.

Conclusion Barbo:

In this hands-on-activity, I learned that how an algorithm grows with input size matters more than how fast it runs on small inputs. Counting operations in the worst case lets you predict that growth, and experiments can confirm it. The results matched the theory. diff and unique are $O(n^2)$, so when n grew 100x, time grew about 10,000x. Binary search is O(log n), so it was almost instant even at 100,000 students. rpower did n multiplications (O(n)) while brpower did only about log n, so at n = 20000 it did 20 multiplications instead of 20000. I think I did alright in this hands-on-activity because I was really confused at the start on what to do but I figured it out as I went on and the theory and results ended up matching. To improve, I want to measure time more precisely, since small runs showed 0 μs, and get better at understanding what an activity asks before I start.

Conclusion Paraiso:

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, Code assistance, Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to help us in our grammar correction, it also helped us in debugging some complicated codes, and was used to explain some concepts.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.